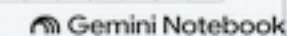
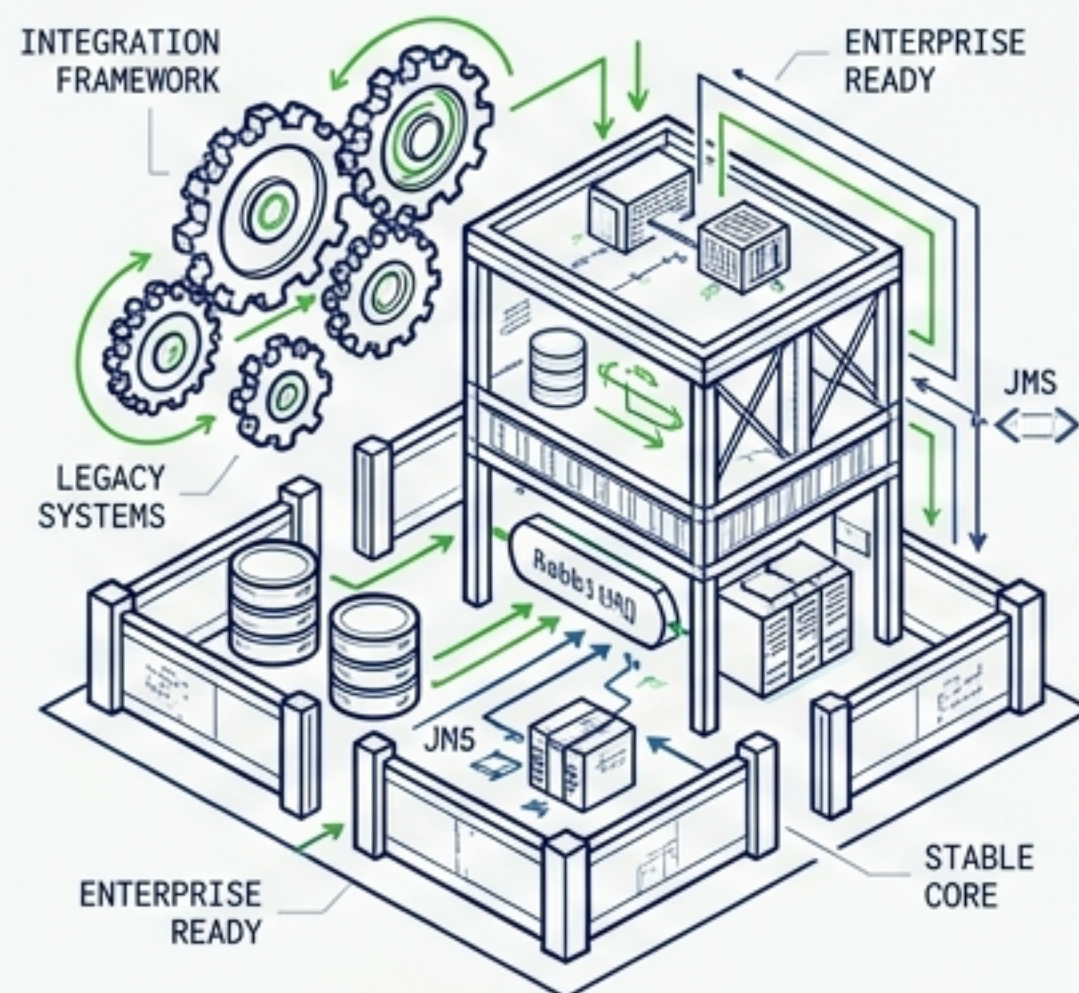


Guía Definitiva de Arquitectura: Modelos de Ejecución, GraalVM y Virtual Threads.

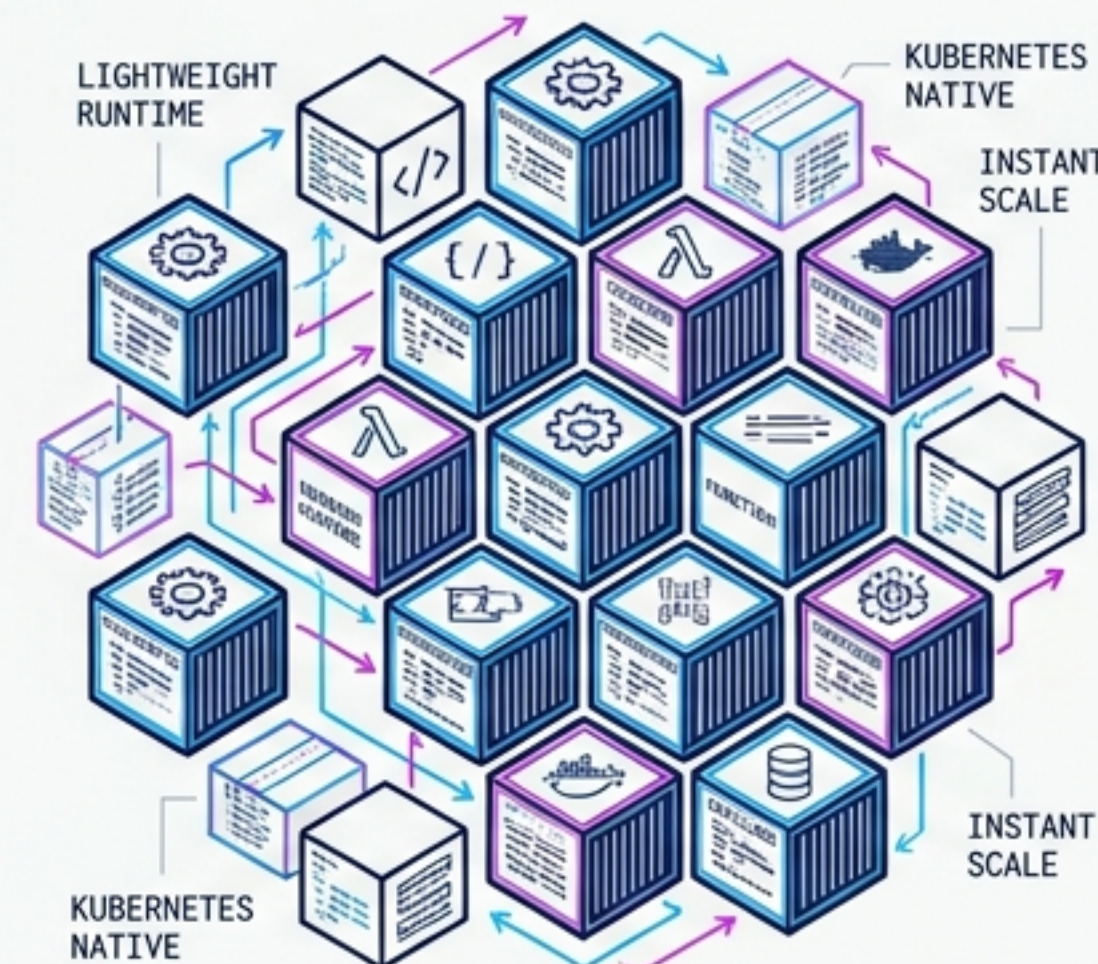


El Dilema Cloud-Native: No hay un ganador absoluto

Ecosistema Masivo



Agilidad Contenerizada



Contexto del Equipo:

¿Dominio previo de Spring o disposición para aprender CDI/Panache?

EXPERTISE OVERHEAD | LEARNING CURVE



Entorno de Ejecución:

¿Servidores tradicionales de larga vida o clústeres K8s con alta densidad de pods?

INFRASTRUCTURE MATURITY | RESOURCE CONSTRAINTS



Requisitos de Rendimiento:

¿Importa más el throughput sostenido o el tiempo de cold-start?

LATENCY SENSITIVITY | OPERATIONAL METRICS

Matriz de Diagnóstico Ejecutivo

Spring Boot 4		Quarkus 3
Convención sobre configuración, ecosistema enorme.	Filosofía	Supersonic Subatomic, optimizado para K8s.
★★★★★ (Estándar de facto)	Madurez	★★★★☆ (Crecimiento agresivo)
Servlet (MVC) o Reactor (WebFlux)	Modelo Base	REST reactivo (Vert.x) + CDI
2-5 s típico 	Arranque (JVM)	0.5-2 s típico 
200-400 MB 	Memoria (RSS reposo)	80-150 MB 
Posible (Spring AOT), más complejo.	GraalVM (Native)	Primera clase (mvn package -Dnative)
Baja (si ya conoces Spring)	Curva de Aprendizaje	Media (CDI, Panache, extensiones)

¿Cuándo elegir qué framework?

Territorio Spring Boot 4

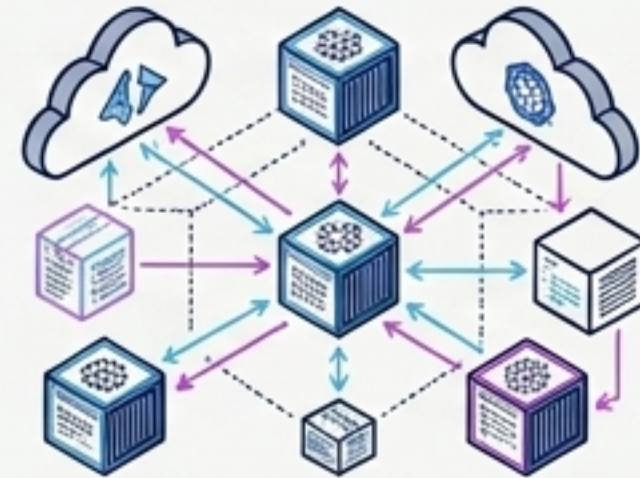


Contextos Ideales

- El equipo ya domina el ecosistema **Spring**.
- Migraciones de monolitos existentes (menor fricción).
- Necesidad integral de **Spring Cloud** (Gateway, Config, Eureka).
- Integraciones enterprise pesadas (Kafka, JMS, SAML, Batch).

Nota: Ideal cuando un arranque de 3 segundos no es crítico (pods de larga vida).

Territorio Quarkus 3

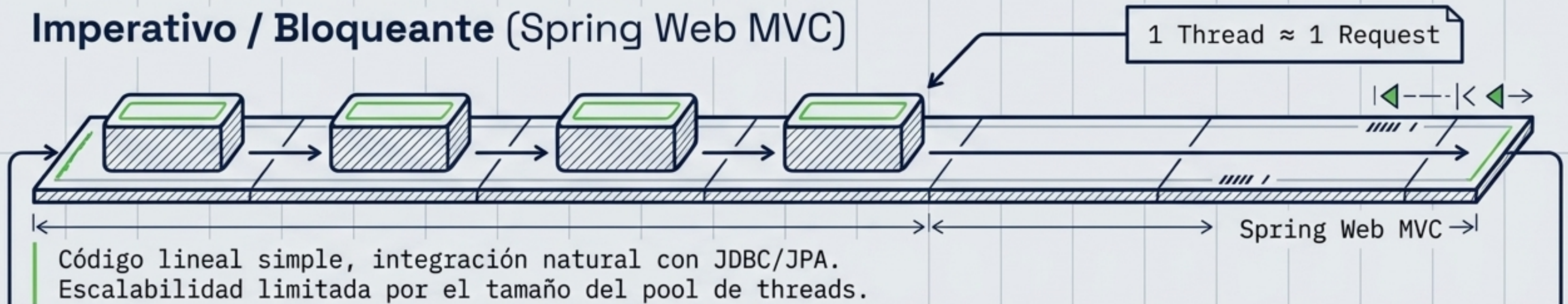


Contextos Ideales

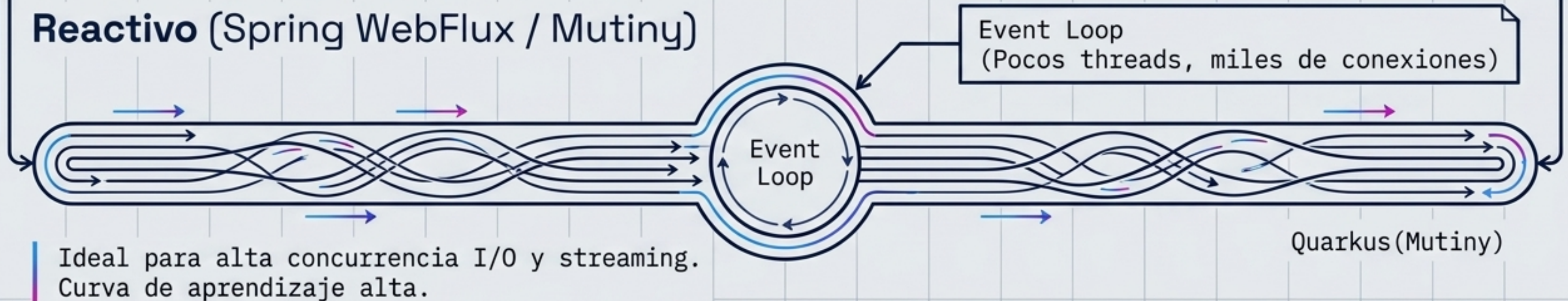
- Despliegue masivo en **Kubernetes** (el costo es memoria x réplicas).
- Arquitecturas **Serverless/Knative** donde el **cold-start** es vital.
- Microservicios extremadamente pequeños y focalizados.
- Prioridad en **Native Image** sin fricciones.

Evolución del Modelo de Programación

Imperativo / Bloqueante (Spring Web MVC)

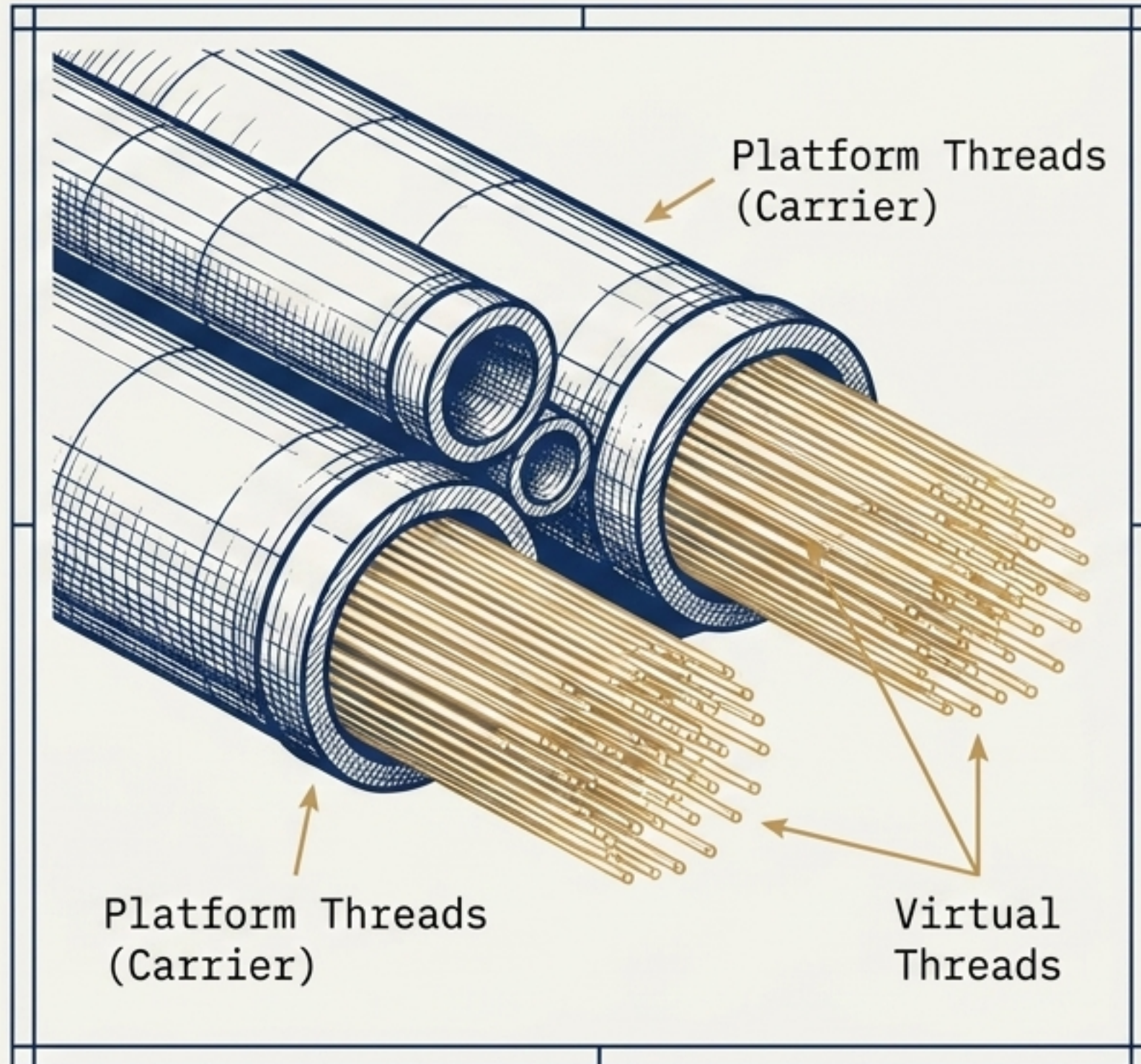


Reactivo (Spring WebFlux / Mutiny)



Regla Práctica: No uses WebFlux solo por moda. Si tu stack de persistencia es JDBC bloqueante, es un anti-patrón sin boundedElastic.

El Cambio de Paradigma: Virtual Threads (Java 21+)



La Promesa

Escribir código **imperativo** (bloqueante) tradicional con **escalabilidad** cercana a la **reactiva**.

El Mecanismo

Miles de requests concurrentes sin saturar el pool. Cuando un thread se bloquea (ej. I/O en BD), el **Carrier Thread** se libera para ejecutar otro **Virtual Thread**.

El Resultado

Alta **escalabilidad** I/O sin la complejidad de Mono/Flux.

Implementando Virtual Threads: Fricción Cero

Spring Boot 4

```
spring.threads.virtual.enabled=true
```

Cero cambios en el código. Tomcat abandona su pool de platform threads y utiliza un `VirtualThreadPoolExecutor`. El `@RestController` es idéntico a MVC.

Quarkus 3

```
@RunOnVirtualThread  
@GET  
@Path("/products")
```

Protege el event loop de Vert.x. Marca explícitamente los métodos bloqueantes (Panache/JDBC) para que el runtime los asigne a un thread virtual de Loom.

Experiencia de Desarrollo y Persistencia

Reducción de Boilerplate (Persistencia)

Spring Data JPA

```
interface ProductRepository extends  
JpaRepository<Product, Long>
```

Requiere inyección del repository.



Quarkus Hibernate Panache

```
Product.findByTenant(id);
```

Patrón Active Record. La entidad es el repositorio. Uso directo.

Quarkus Dev Mode

Live Reload sub-segundo.

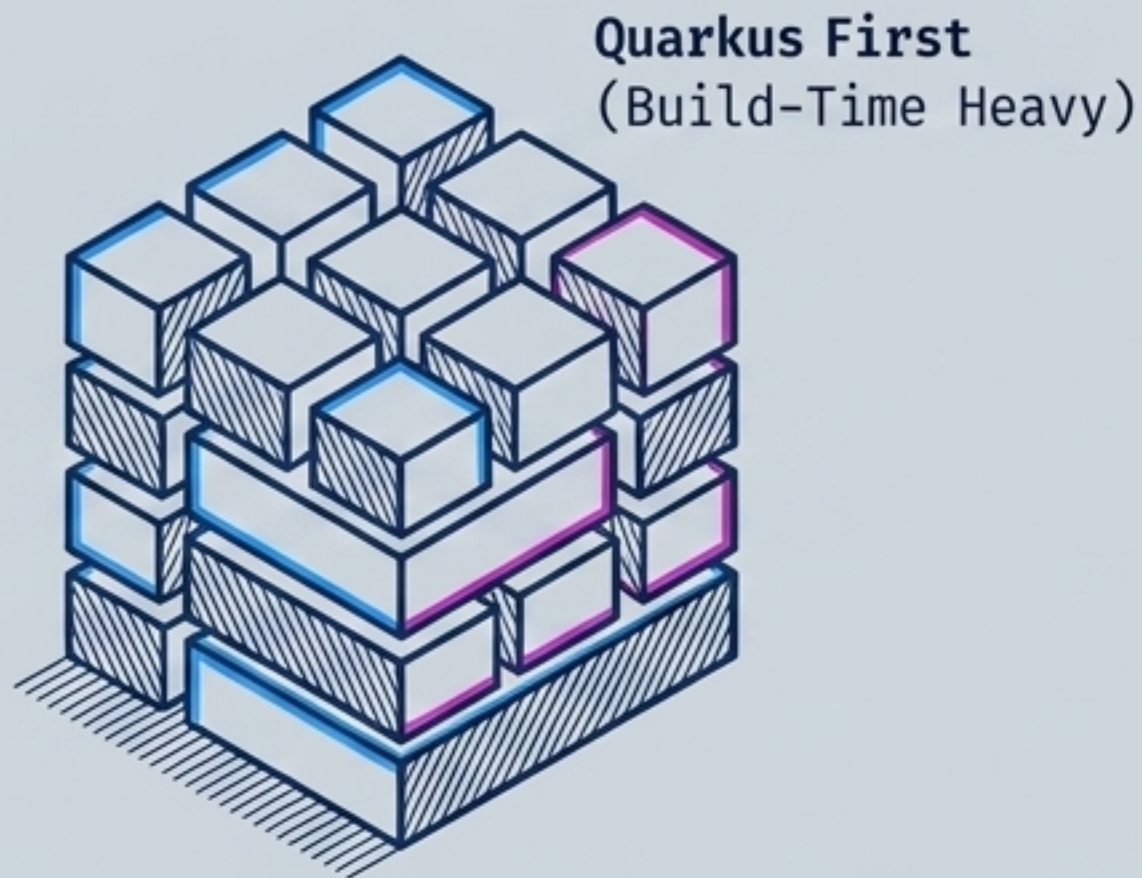
Cambias un @Path o entidad, y Quarkus recompila al vuelo sin reiniciar la JVM completa.



La Revolución AOT (Ahead-of-Time) y GraalVM

Build-Time (Compilación)

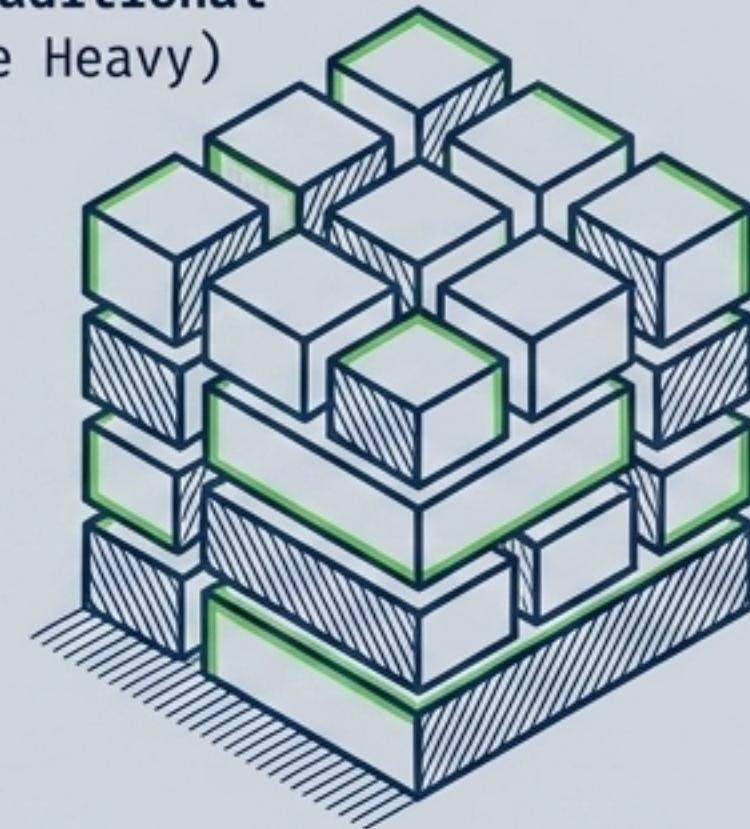
Run-Time (Ejecución)



- Generación de código y configuración fija.
- Eliminación de código muerto.
- Binario nativo de ~50-80 MB que arranca en milisegundos.

```
mvn package -Dnative
```

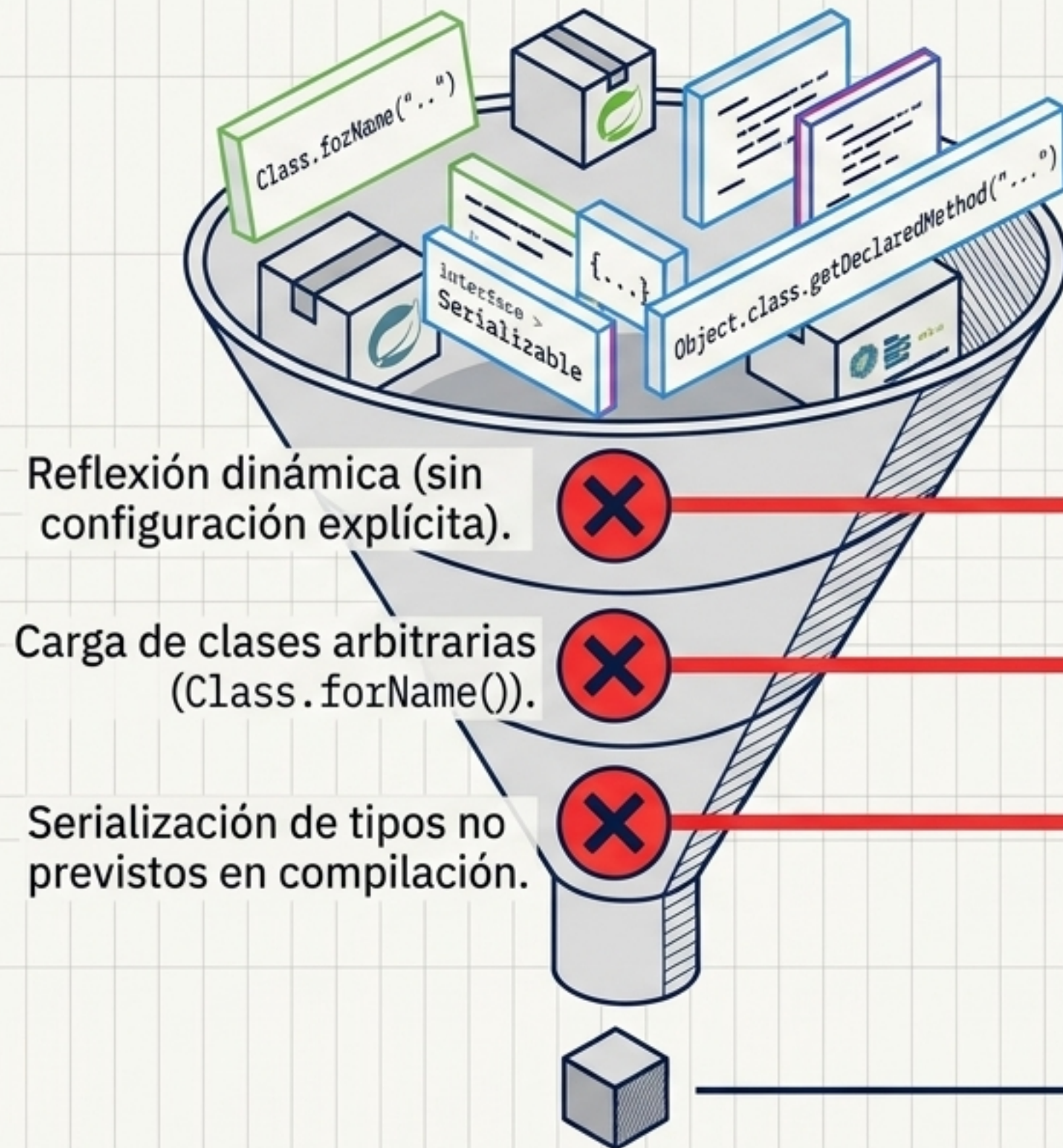
Spring Traditional
(Run-Time Heavy)



- Escaneo dinámico del classpath.
- Auto-configuración reflexiva al arrancar.

(Nota: Spring Boot 3/4 AOT mejora esto notablemente, pero Quarkus fue diseñado con esta arquitectura desde el día 1).

Limitaciones del Mundo Cerrado (GraalVM)



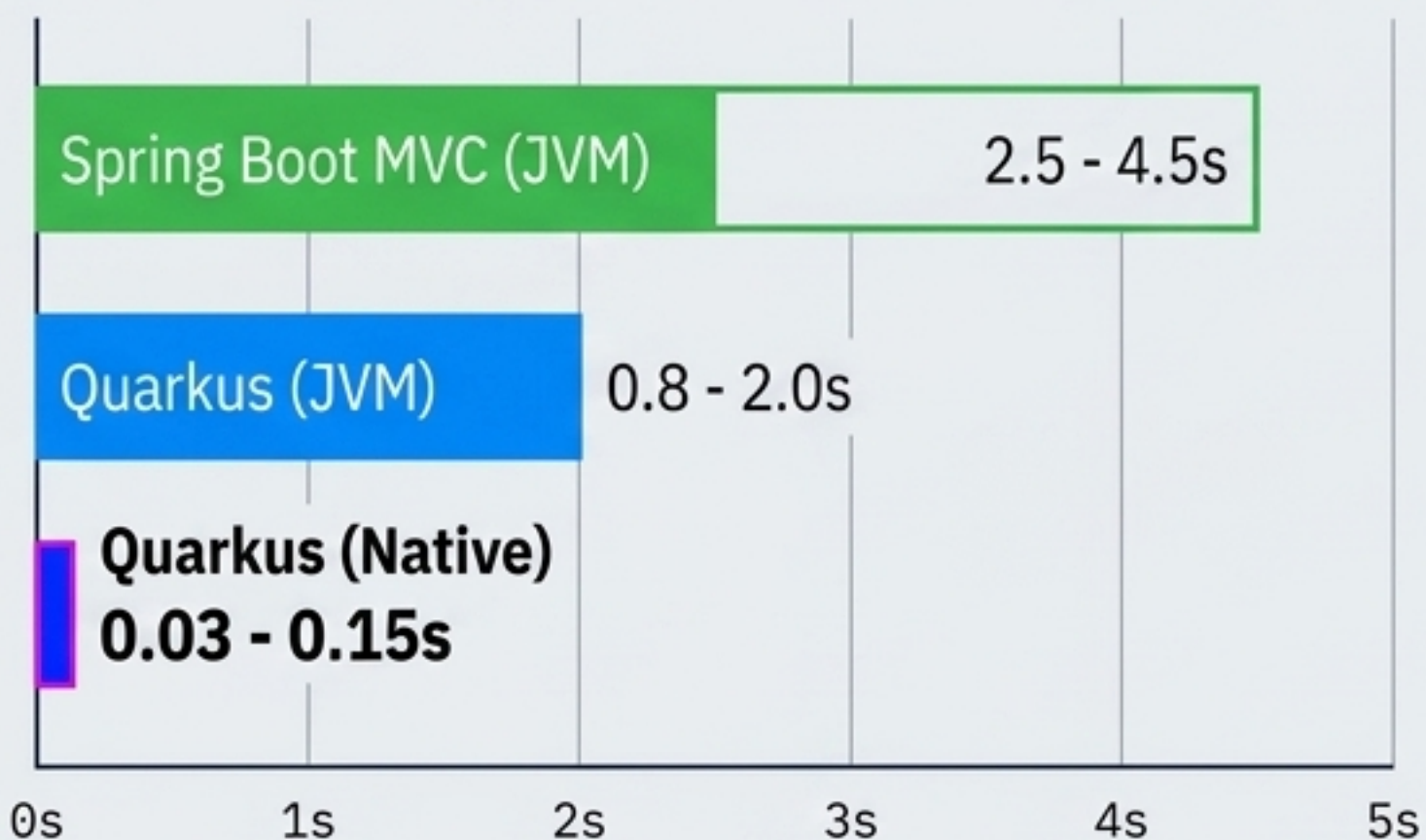
El binario nativo NO puede cargar clases desconocidas en runtime.

Solución Quarkus

Sus extensiones (JPA, REST, Jackson) ya generan automáticamente el `reflect-config.json` necesario, ocultando esta complejidad al desarrollador.

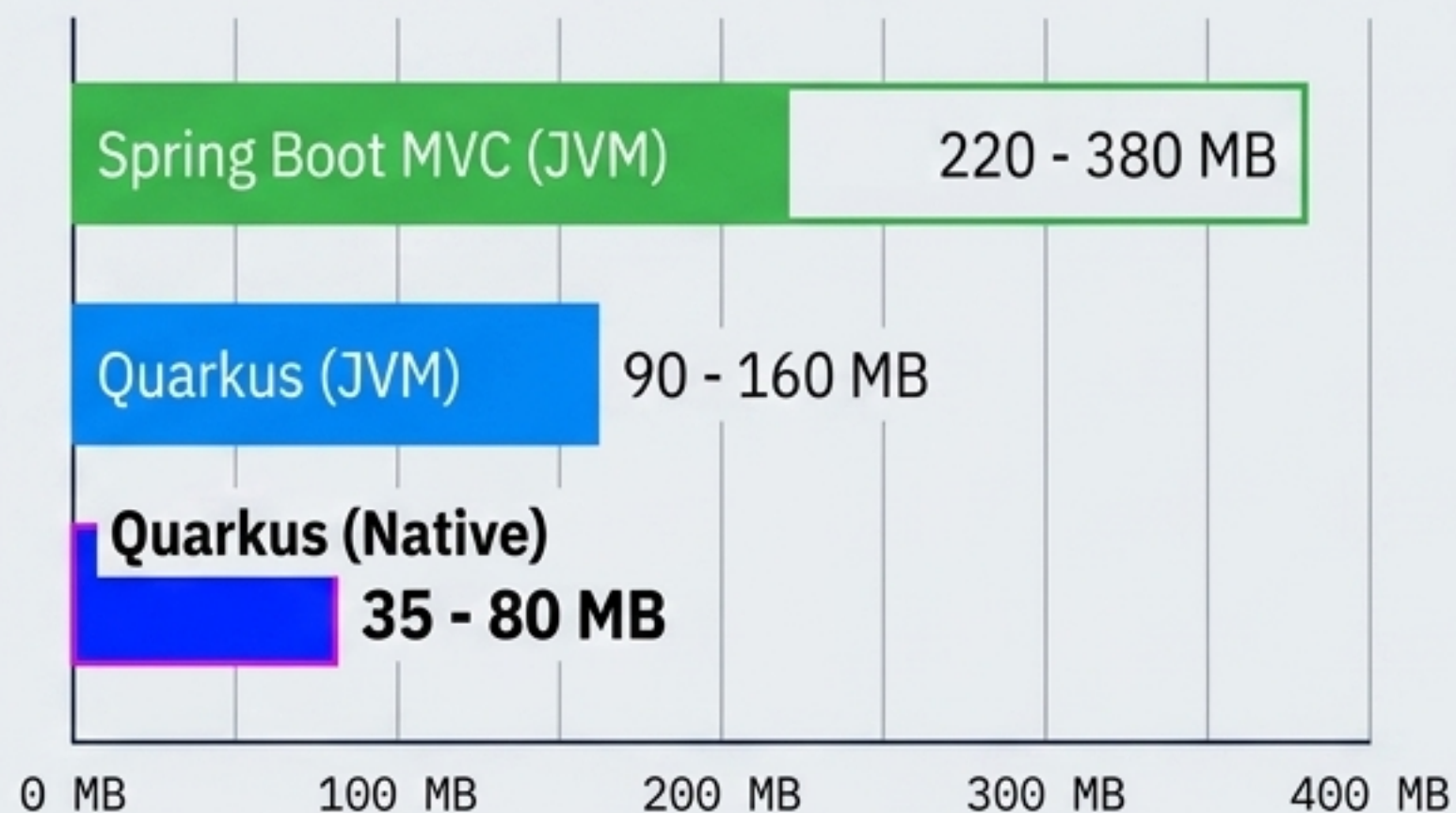
Telemetría: Tiempos de Arranque y Memoria

Startup Time (Segundos hasta HTTP 200)



Tiempo para primera petición HTTP

Memoria RSS en Reposo (MB)

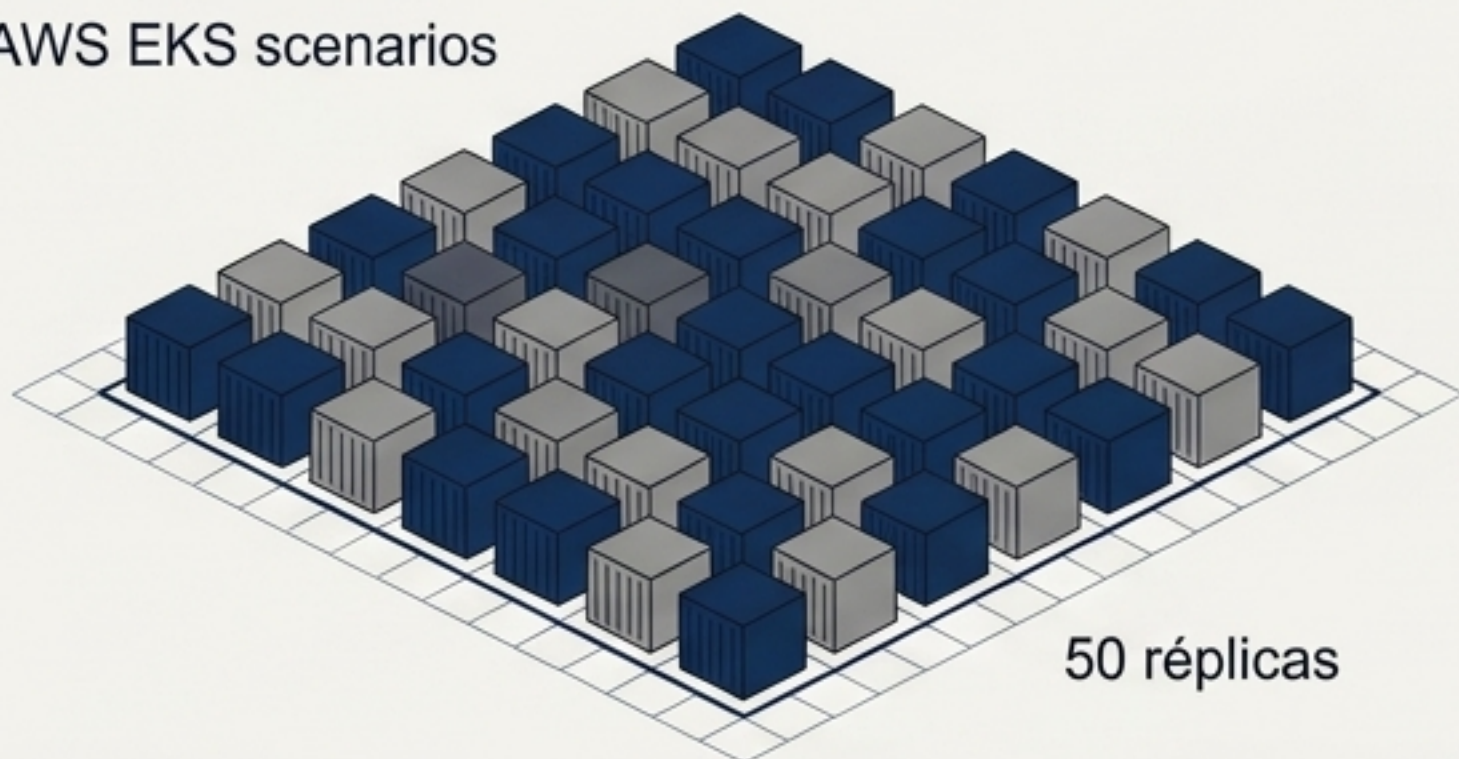


Memoria residente sin carga (RSS)

Los Virtual Threads NO reducen el tiempo de arranque ni la memoria en reposo; su beneficio radica puramente en el throughput bajo carga concurrente.

Impacto Económico en Producción (Kubernetes)

AWS EKS escenarios

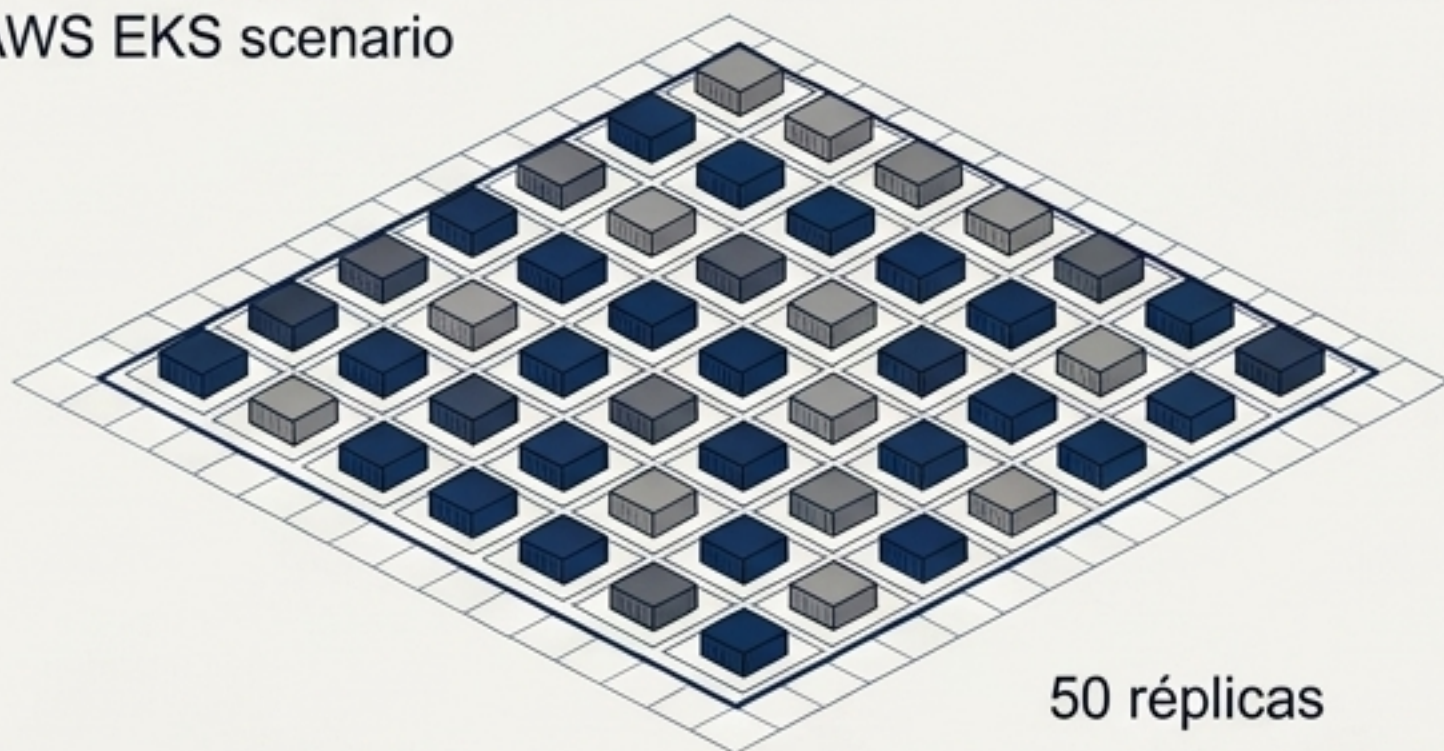


Ruta JVM Clásica

$$50 \text{ pods} \times 300 \text{ MB} = \mathbf{15 \text{ GB RAM}}$$

Requiere nodos AWS más grandes y costosos.

AWS EKS escenario



Ruta Native GraalVM

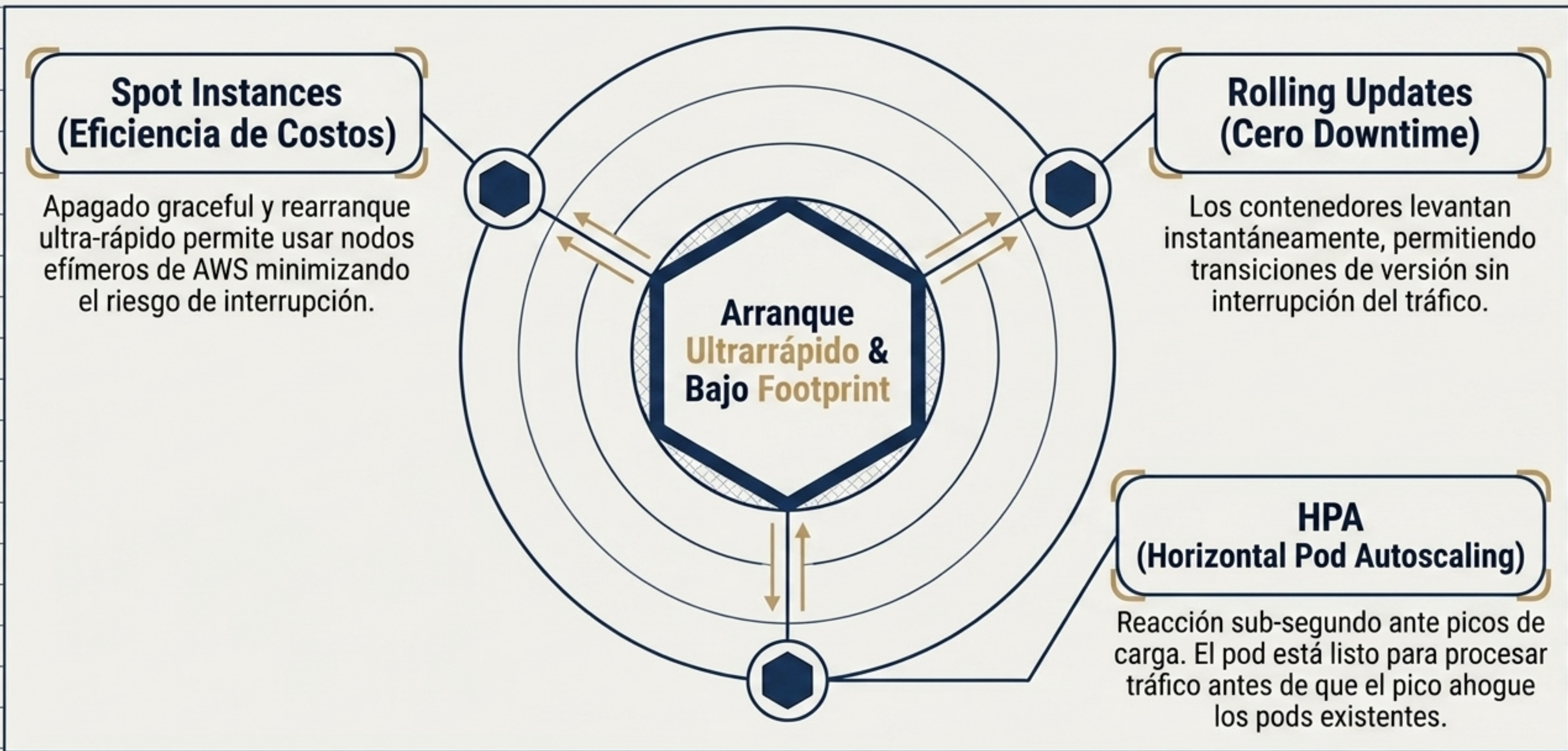
$$50 \text{ pods} \times 60 \text{ MB} = \mathbf{3 \text{ GB RAM}}$$

Ahorro masivo de infraestructura. Mayor densidad por nodo.



Advertencia: Las builds nativas tardan significativamente más en los pipelines de CI/CD y exigen metodologías de debugging más complejas.

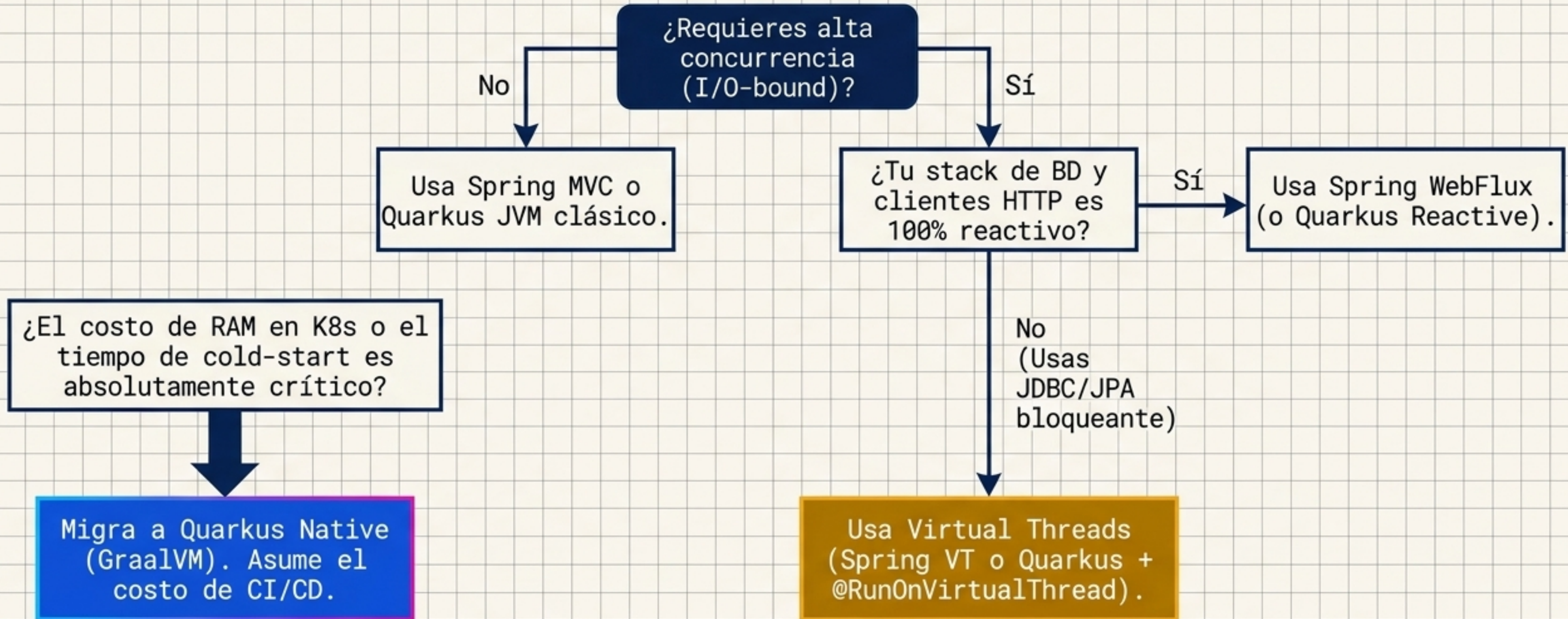
Factor 9 y Orquestación Avanzada en K8s



Síntesis Definitiva: Los 5 Stacks

Puerto	Stack	Thread Model	Ideal Cuando...
8081	Spring MVC	Pool de Platform Threads	CRUD simple, concurrencia media, ecosistema Spring puro.
8082	Spring WebFlux	Event Loop Reactivo	Streaming, arquitecturas I/O reactivas end-to-end. (Curva alta).
8083	Quarkus JVM	Worker/Event Loop	Equilibrio entre ecosistema JVM y bajo consumo de RAM.
8084	Spring MVC + VT	Virtual Threads	Código JDBC/imperativo con necesidad de alta concurrencia. Equipo experto en Spring.
8085	Quarkus + VT	Virtual Threads	Código imperativo con altísima concurrencia, optimizado para K8s y Native Image futuro.

Árbol de Decisión Arquitectónica



La arquitectura es el arte de gestionar trade-offs.
Elige el stack que resuelva tu mayor cuello de botella actual.